

Coach11

Auditoria Técnica Completa

Arquitetura | Segurança | Performance | Qualidade

Data: 18/03/2026

Dominio: coach11.app

Stack: Next.js 16 + React 19 + Supabase + TypeScript

Índice

1. Resumo Executivo
2. Arquitetura UML do Projeto
3. Erros e Problemas Identificados
4. Auditoria de Segurança
5. Análise de Performance e Escalabilidade
6. Refactoring Necessário
7. Smoke Test / Community Test
8. Migração de Domínio (vercel.app -> coach11.app)
9. Plano de Ação por Sprints

1. Resumo Executivo

O Coach11 e uma aplicacao web/PWA para gestao de escaloes de futebol em clubes. Esta auditoria avalia o estado actual do projecto em 6 dimensoes: arquitectura, seguranca, performance, qualidade de codigo, testes e preparacao para producao.

Scorecard Geral

Area	Score	Observacao
Seguranca	8.5/10	Forte - CSP, RLS, sanitizacao, rate limiting
Arquitectura	7.5/10	Solida age_group-first, domain boundaries
Qualidade Codigo	8/10	0 erros TS, 0 any, boa tipagem
Performance	6/10	Todas paginas client-side, sem loading states
Testes	4/10	17 test files, 0 E2E, 0 componentes
Producao-ready	6.5/10	Dominio OK, PWA OK, falta CI/CD robusto

PONTOS FORTES: Zero erros TypeScript, zero uso de 'any', security headers robustos, domain boundaries com guardrails automatizados, RPC parametrizado, sanitizacao de HTML, rate limiting implementado.

PONTOS A MELHORAR: 22 paginas todas client-rendered, sem loading.tsx/error.tsx, cobertura de testes insuficiente para escalar, 66% das API routes sem validacao Zod, rate limiting in-memory (nao distribuido).

2. Arquitectura UML do Projeto

2.1 Visao Geral da Estrutura

O projecto segue o Next.js App Router com a seguinte organizacao:

```
src/
  app/      -> Rotas (pages, layouts, API)
  (auth)/   -> Login, registo, convites
  (dashboard)/ -> Area autenticada principal
  api/      -> 62 API routes REST
  public/   -> Paginas publicas por token
  components/ -> 20 modulos de UI organizados por dominio
  lib/      -> Logica de negocio, hooks, utils, services
  types/    -> Definicoes TypeScript (database.ts manual)
```

2.2 Diagrama de Componentes

CAMADA DE APRESENTACAO (Client Components)

```
|-- Layout (Sidebar, Navigation, TopBar)
|-- Dashboard (DashboardPage, StatsCards, UpcomingEvents)
|-- Players (PlayerList, PlayerCard, PlayerForm)
|-- Games (GameList, GameDetail, LiveGame, Convocation)
|-- Trainings (TrainingList, AttendanceGrid)
|-- Calendar (CalendarView, EventModal)
|-- Statistics (StatsTable, PDFExport)
|-- Settings (TeamSetup, AccountSettings)
|-- PWA (PWAProvider, InstallPrompt, PushNotifications)
```

CAMADA DE LOGICA (Hooks + Services)

```
|-- useLiveGameState (1668 linhas - necessita refactoring)
|-- useTeamSetup, useGameDetailData, useCalendarData
|-- calendar-events.service.ts (795 linhas)
|-- public-share.ts, rate-limit.ts, auth/beta-access.ts
```

CAMADA DE DADOS (Supabase)

```
|-- Supabase Client (SSR + Browser)
|-- RPC Functions (7 stored procedures)
|-- Row Level Security (RLS policies)
|-- Storage (logos, imagens de eventos)
|-- Auth (email/password + recovery)
```

2.3 Modelo de Dados (Entidades Principais)

Entidade	Descricao
Profile	Utilizador autenticado (coordenador, treinador, jo
AgeGroup	Escalao - raiz funcional do sistema (ex: Sub-15)
Player	Jogador pertencente a um escalao
AgeGroupStaff	Staff tecnico do escalao (coach, assistant_coach)
Game	Jogo agendado/live/finalizado
Convocation	Convocatoria para jogo (draft/confirmed/closed)
GameEvent	Evento de jogo (golo, cartao, substituicao)
Training	Sessao de treino
Attendance	Presenca em treino ou jogo
Competition	Liga, taca ou amigavel
Message	Mensagem de equipa com tracking de leitura
Notification	Push notification e in-app
KitConfig	Configuracao de equipamentos por kit

2.4 Relacoes Chave

AgeGroup (1) --> (*) Player
 AgeGroup (1) --> (*) AgeGroupStaff
 AgeGroup (1) --> (*) Game
 AgeGroup (1) --> (*) Training
 AgeGroup (1) --> (*) Competition
 Game (1) --> (*) Convocation --> (*) Player
 Game (1) --> (*) GameEvent
 Training (1) --> (*) Attendance --> (1) Player
 Profile (1) --> (*) AgeGroupStaff
 Profile (1) --> (1) AgeGroup (as coordinator)

2.5 Evolucao Architectural Prevista

FASE ATUAL: age_group-first (escalao unico por coordenador)

FASE 2 (Proxima): Club-first hierarchy
 Club --> AgeGroupCategory --> AgeGroup
 Novo: ClubAdmin, cross-age-group visibility

FASE 3: Multi-tenant
 Platform --> Club --> AgeGroup
 Subscricoes, billing, onboarding autonomo

3. Erros e Problemas Identificados

3.1 TypeScript e Lint

- OK** 0 erros TypeScript (tsc --noEmit limpo)
- OK** 0 erros de lint (apenas 3 warnings)
- OK** 0 uso de 'any', @ts-ignore, @ts-expect-error

3.2 Warnings a Resolver

- BAIXO** getLineupLabel nao utilizado (summary/page.tsx:110)
- BAIXO** 2x em vez de next/image (staff/page.tsx, ClubLogoUpload.tsx)
- BAIXO** 11 eslint-disable (7 exhaustive-deps + 4 set-state-in-effect)

3.3 Problemas de Producao

- MEDIO** Build falha sem SUPER_COORDINATOR_EMAIL env var

O ficheiro beta-access.ts faz throw no module load time quando a variavel de ambiente nao existe. Isto impede builds locais e CI sem configuracao completa.

- MEDIO** console.info('[auth.debug]') em PWAProvider.tsx em producao

~5 chamadas de debug logging que executam em cada mudanca de estado de auth e resume do foreground. Devem ser gated por NODE_ENV === 'development'.

4. Auditoria de Seguranca

4.1 Autenticacao e Autorizacao

- OK** Todas as API routes verificam `supabase.auth.getUser()`
- OK** Admin routes usam `getSuperUserAccess()` dedicado
- OK** RPC functions parametrizadas (0 vectores de SQL injection)
- OK** Open redirect protegido via `sanitizeNextPath()`
- OK** Password minimo 10 caracteres enforced
- INFO** Sem `middleware.ts` - auth per-route (funcional mas nao centralizado)

4.2 Headers de Seguranca

- OK** CSP configurado sem `unsafe-eval`
- OK** X-Frame-Options: DENY
- OK** Strict-Transport-Security: max-age=63072000
- OK** Permissions-Policy: camera=(), microphone=(), geolocation=()

4.3 Input Validation

- MEDIO** 34% das API routes usam Zod (21/62)

41 API routes (~66%) nao tem validacao explicita Zod. Usam validacao manual que e mais propensa a erros. Recomenda-se migrar para Zod progressivamente.

4.4 Rate Limiting

- OK** Rate limiting implementado para invites, location, redeem

- MEDIO** Rate limiting in-memory (per-instance em serverless)

Em Vercel serverless, cada instancia tem o seu rate limiter independente. Para proteccao distribuida real, migrar para Upstash Redis ou Vercel KV.

4.5 XSS e Sanitizacao

- OK** `dangerouslySetInnerHTML` usado 1x com sanitizacao completa
- OK** HTML gerado via `renderNotesToHtml()` com escape de caracteres

4.6 Suite de Testes de Seguranca

- OK** `security-fixes.test.ts` com 19 testes automatizados

Valida: CSP sem `unsafe-eval`, password min 10 chars, sem hardcoded secrets, score max cap, correlationId hidden em producao, validacao de nomes.

5. Analise de Performance e Escalabilidade

5.1 Renderizacao

ALTO 22/22 paginas sao 'use client' - zero SSR

TODAS as paginas do dashboard sao client-rendered. Isto significa:

- Maior bundle JS enviado ao browser
- Sem beneficio de streaming SSR do Next.js
- SEO limitado (menos relevante para dashboard, mas critico para paginas publicas)
- Maior Time to Interactive (TTI)

Recomendacao: Migrar paginas para Server Components com 'use client' apenas nos componentes interactivos. Priorizar dashboard, calendar, statistics.

5.2 Loading States e Error Boundaries

ALTO 0 ficheiros loading.tsx encontrados

ALTO 0 ficheiros error.tsx (apenas global-error.tsx)

Sem loading.tsx, o utilizador ve um ecrã branco durante a navegacao. Sem error.tsx por segmento, erros em sub-paginas crashes toda a app.

Recomendacao: Adicionar loading.tsx com skeletons em (dashboard)/ e cada sub-rota. Adicionar error.tsx em (dashboard)/ e (auth)/.

5.3 Caching

OK React Query (TanStack) para caching client-side

OK unstable_cache para paginas publicas (ISR)

OK Cache-Control headers bem configurados para assets

MEDIO Sem caching server-side para API routes autenticadas

5.4 Bundle e Dependencias

MEDIO leaflet importado globalmente (pode ser lazy-loaded)

MEDIO jspdf importado globalmente (so usado na exportacao)

Bibliotecas como leaflet e jspdf devem usar dynamic import para reduzir o bundle inicial. Exemplo: const leaflet = await import('leaflet').

5.5 Optimizacoes React

OK 111 usos de useMemo/useCallback em 27 ficheiros

Boa adopcao de memoizacao, especialmente em useLiveGameState.ts (26 usos). Manter esta pratica ao refactorar

componentes.

5.6 Paginação

OK 23 API routes implementam paginação

Boa cobertura de paginação nas endpoints de listagem.

6. Refactoring Necessario

6.1 Ficheiros Grandes (Prioridade Alta)

Ficheiro	Linhas	Accao Recomendada
useLiveGameState.ts	1,668	Dividir em sub-hooks por responsabilidade
games/[id]/summary/page.tsx	952	Extrair componentes de sumario
competitions/page.tsx	928	Separar CRUD de display
team/page.tsx	909	Extrair tabs em componentes
staff/page.tsx	840	Separar lista de formularios
players/page.tsx	813	Extrair PlayerForm e PlayerList
calendar-events.service.ts	795	Dividir por tipo de evento
games/page.tsx	680	Extrair GameCard e GameFilters
LandingPage.tsx	664	Dividir em seccoes
settings/page.tsx	652	Extrair cada tab em componente

6.2 Validacao de API Routes

ALTO 41 API routes sem validacao Zod (66% do total)

Criar schemas Zod para todas as API routes. Priorizar routes que aceitam POST/PUT/PATCH. Padrao recomendado: `schema.safeParse(body)` no inicio de cada handler.

6.3 Tipos de Base de Dados

MEDIO database.ts e manual (298 linhas) - risco de dessincronia

Usar 'supabase gen types typescript' para gerar tipos automaticamente. Manter database.ts como facade que re-exporta os tipos gerados.

6.4 Camada de Repositorio

MEDIO Apenas 3 ficheiros em src/lib/repositories/

57+ ficheiros usam Supabase directamente. Consolidar queries repetidas em repositorios por dominio: PlayerRepository, GameRepository, etc.

6.5 Blocos catch {} Vazios

ALTO 104 blocos catch {} vazios em todo o codebase

104 blocos catch que engolem erros silenciosamente. Torna o debugging extremamente dificil. Adicionar pelo menos `console.error` ou `toast` em cada um. Priorizar os que estao em componentes client-side e API routes.

6.6 API Routes sem respondInternalServerError

MEDIO 3 API routes sem error reporting padronizado

Falta respondInternalServerError + Sentry em:

- api/calendar/events/route.ts
- api/invite/redeem/route.ts
- api/waitlist/route.ts

Todas as outras ~58 routes usam o padrao correctamente.

6.7 Chamadas Supabase Directas em Pages

MEDIO 6 dashboard pages com supabase.from() directo

Pages que devem migrar para hooks/repositories:

- competitions/page.tsx
- dashboard/page.tsx
- join/page.tsx
- messages/page.tsx
- notifications/page.tsx
- settings/page.tsx

6.8 Padroes de Data Fetching Inconsistentes

MEDIO 3 padroes diferentes de fetch no client-side

O codebase usa 3 padroes diferentes:

1. fetch() directo com res.json().catch(() => ({}))
2. apiFetch<T>() wrapper (lib/http/apiFetch.ts)
3. supabase.from() directo em pages

Recomendacao: Standardizar em apiFetch para client->API, mover Supabase para server components ou camada de repositorio.

6.9 eslint-disable Comments

BAIXO 11 eslint-disable comments (7 exhaustive-deps)

Revisar cada supressao - muitas podem ser resolvidas com useCallback ou refs. As restantes devem ter comentarios explicativos.

6.10 Environment Variables

MEDIO 20+ env vars sem .env.example documentado

Variaveis criticas dispersas pelo codebase sem documentacao centralizada. Nota: SUPABASE_SERVICE_ROLE_KEY tem 3 nomes fallback diferentes - consolidar. Criar .env.example com todas as variaveis e descricoes.

7. Smoke Test / Community Test

7.1 Resultados

- OK

17 ficheiros de teste, 66 testes unitarios - TODOS PASSAM
- OK

Lint: 0 erros, 3 warnings
- OK

Architecture guard: 313 ficheiros, PASSED
- MEDIO

Build: FALHA sem env vars (SUPER_COORDINATOR_EMAIL)

7.2 Cobertura de Testes Existente

Ficheiro	Testes	Area
security-fixes.test.ts	19 testes	Validacao de seguranca
sanitize-next.test.ts	6 testes	Open redirect protection
canonical-app-url.test.ts	3 testes	URL canonica
convocation-editor.test.ts	Varios	Editor de convocatorias
live-kickoff.test.ts	Varios	Inicio de jogo live
live-persistence.test.ts	Varios	Persistencia de estado
public-live/convocation.test.ts	Varios	Paginas publicas
osm/google-place-id.test.ts	8 testes	Providers de localizacao

7.3 Lacunas Criticas de Testes

- CRITICO

0 testes E2E (Playwright/Cypress)
- CRITICO

0 testes de componentes React
- ALTO

0 testes de integracao para API routes
- MEDIO

Sem .env.example para onboarding de developers
- MEDIO

Sem vitest.config.ts (defaults usados)

7.4 Community Test Recomendado

Fluxos criticos que DEVEM ter testes E2E:

1. Login -> Dashboard -> Ver escalao
2. Criar jogo -> Convocatoria -> Confirmar -> Live game
3. Live game -> Golos/Cartoes -> Finalizar -> Sumario
4. Criar treino -> Marcar presencas
5. Convidar staff -> Aceitar convite -> Acesso ao escalao
6. Pagina publica -> Verificar dados visiveis
7. PWA -> Push notification -> Click -> Navegacao

8. Exportar PDF de estadísticas

8. Migração de Domínio

8.1 Estado Atual

- OK** Código-fonte 100% migrado para coach11.app
- BAIXO** 1 referência restante em README.md (exemplo curl)

8.2 Ações Necessárias

- Atualizar README.md:53 - curl URL de coach11.vercel.app para coach11.app
- Verificar configuração DNS no Vercel (domínio custom coach11.app)
- Configurar redirect 301 de coach11.vercel.app para coach11.app
- Atualizar Supabase Auth redirect URLs para coach11.app
- Verificar CORS origins no Supabase dashboard
- Atualizar canonical URLs em meta tags (SEO)
- Verificar push notification VAPID_SUBJECT usa coach11.app
- Atualizar quaisquer webhooks externos (Resend, Sentry, PostHog)

9. Plano de Acao por Sprints

Sprint 1: Fundacao e Qualidade (Semana 1-2)

Prioridade: Critico

- Criar .env.example com todas as env vars necessarias
- Fix: beta-access.ts - mover throw para runtime, nao module load
- Adicionar loading.tsx em (dashboard)/ com skeleton screens
- Adicionar error.tsx em (dashboard)/ e (auth)/
- Remover getLineupLabel nao utilizada
- Actualizar README.md com dominio coach11.app
- Configurar Playwright para testes E2E
- Criar 3-5 testes E2E para fluxos criticos (login, criar jogo, live game)
- Gating dos console.info debug em PWAProvider.tsx
- Corrigir catch {} vazios mais criticos (API routes e hooks)
- Adicionar respondInternalServerError a 3 API routes em falta
- Consolidar 3 nomes fallback de SUPABASE_SERVICE_ROLE_KEY

Sprint 2: Performance e Validacao (Semana 3-4)

Prioridade: Alto

- Adicionar Zod validation a API routes POST/PUT (priorizar games, players, staff)
- Corrigir restantes catch {} vazios (104 no total)
- Migrar dashboard page para Server Component
- Lazy-load leaflet e jspdf com dynamic imports
- Substituir por next/image em staff/page.tsx e ClubLogoUpload.tsx
- Refactorar useLiveGameState.ts (1668 linhas) em sub-hooks
- Implementar rate limiting distribuido (Upstash Redis)
- Adicionar mais 5-10 testes E2E para fluxos secundarios

Sprint 3: Refactoring e Arquitectura (Semana 5-6)

Prioridade: Medio

- Refactorar pages grandes: summary, competitions, team, staff, players (>800 linhas)
- Expandir camada de repositorio (PlayerRepo, GameRepo, TrainingRepo)
- Mover 6 pages com supabase.from() directo para hooks/repositories
- Standardizar data fetching em apiFetch (eliminar 3 padroes diferentes)
- Gerar tipos Supabase automaticamente (supabase gen types)
- Revisar e resolver 7 eslint-disable exhaustive-deps
- Adicionar middleware.ts centralizado para auth (opcional mas recomendado)

- Adicionar testes unitários para repositories e services

Sprint 4: Escalabilidade Multi-Clube (Semana 7-8)

Prioridade: Estratégico

- Modelo de dados: Club > AgeGroupCategory > AgeGroup
- Dashboard de clube com vista panorâmica de escalões
- Role 'Club Admin' com acesso cross-escalão
- Switching de contexto entre escalões (sidebar dropdown)
- Expandir roles de staff (estagiário, preparador físico, delegado, etc.)
- Cross-escalão: convocatória de jogadores de outros escalões
- Testes E2E para novos fluxos multi-escalão

Sprint 5: Polish e Produção (Semana 9-10)

Prioridade: Importante

- Pipeline CI/CD completo (lint + tsc + test + build)
- Monitoring e alertas (Sentry alerts, uptime checks)
- Performance audit com Lighthouse (target: >80 em todas as métricas)
- Documentação técnica para novos developers
- Security penetration testing manual
- Load testing para cenários multi-clube (50+ clubes)
- Portal de pais/encarregados (vista read-only)
- Avaliação de jogadores (fichas individuais)

Resumo de Accoes Imediatas (Top 10)

#	Severidade	Accao
1	CRITICO	Criar .env.example e fix beta-access.ts build erro
2	CRITICO	Adicionar loading.tsx e error.tsx boundaries
3	CRITICO	Configurar Playwright e criar primeiros testes E2E
4	ALTO	Corrigir 104 catch {} vazios (erros silenciosos)
5	ALTO	Adicionar Zod validation a 41 API routes
6	ALTO	Refactorar useLiveGameState.ts (1668 linhas)
7	ALTO	Migrar paginas para Server Components
8	MEDIO	Standardizar data fetching (3 padroes -> 1)
9	MEDIO	Implementar rate limiting distribuido
10	MEDIO	Expandir camada repositório + gerar tipos Supabase

Este relatorio foi gerado automaticamente com base na analise estatica do codigo-fonte, execucao de testes, e inspecao de configuracoes. Recomenda-se complementar com testes manuais e penetration testing.